

INDU: Short-read mapping

Kristoffer Sahlin

9 januari 2024

Innehåll

Introduction	1
Simplifying assumptions	1
Assignment	2
Proposed approach	3
Reference coverage	3
Output file	3
Test data	4
Criteria	4
The report	4
Data	4
Grading	4

Introduction

Short-read mapping is one of the most fundamental computational problems in bioinformatics. It is needed for a range of bioinformatic tasks such as single-nucleotide polymorphism and structural variation detection, abundance estimation (metagenomics), comparative genomics etc. Typically, short reads are in the form of Illumina sequencing reads (about 50-250nt in length). Read mapping is the computational problem of finding the highest scoring location of a read with respect to a set of reference sequences, e.g., the human chromosomes. The score is typically computed from the number of matches, substitutions, insertions, and deletions between the read and the reference at a given location. In this project, you will develop an algorithm to map reads to references.

Simplifying assumptions

1. We will simplify the problem by ignoring insertions and deletions (indels). That is, any location with an indel can be ignored.
2. We will also assume that all substitutions have the same penalty. Thus, you only need to find the mapping location(s) of a read with the fewest substitutions¹, below referred to as *best mapping location(s)*.
3. Also, you do not need to care about reverse complement matches.

¹This is often referred to as finding the smallest *Hamming distance* (see e.g. https://en.wikipedia.org/wiki/Hamming_distance) between the read and the reference(s).

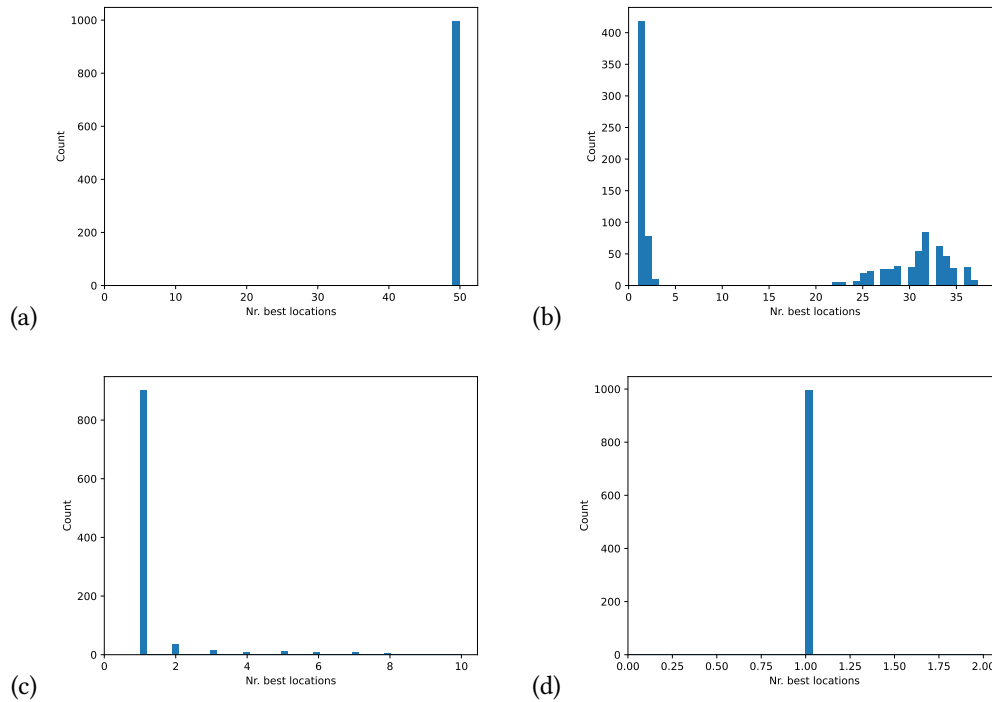


Figure 1: Figures of number of best mapping locations, for datasets with 50 reference sequences with 0% (a), 1% (b), 5% (c), and 10% (d) mutation rates to each other. These datasets are found in the project folder. Your plots do not have to be identical in style, but it is important that you have the correct results. I used 50 bins to plot the histograms.

Assignment

You should write a Python program that

1. reads in a set of references (fasta), and a set of reads (fasta),
2. finds the best locations (there can be several) for each read,
3. Produces three files:
 - All best locations to a file (details on format below)
 - The *reference coverage* for each reference (details on format below)
 - A histogram plot displaying the number of mapping locations (x-axis) per read (see example Figure 1)

The program should be executable from the terminal with arguments. Here is an example call to a program for example dataset 1 (see course webpage):

```
python read_mapping.py --k 3 refs_small.fa reads_small.fa output/
```

where the last mandatory argument is a folder where the output files go. Parameter k should be set much larger for the other datasets (e.g. 30).

The program should also handle common errors; wrong file name/missing file, wrong fasta format, invalid arguments. The error handling for invalid arguments comes for free if you decide to use argparse.

With the help of your program and the data files you can download to your computer, it should be possible to address the question: *Which reference sequence is the most abundant (highest coverage) in the sample?* for each read file. The project requires algorithmic thinking. I have provided some guidance below, but feel free to use any method you can come up with.

Proposed approach

Consider a read of length r with m substitutions. It is easy to show (pigeon principle) that there will be a substitution-free stretch of at least $\lfloor r/m \rfloor$ nucleotides. For example, with $r = 100$ and $m = 3$, we have $\lfloor r/m \rfloor = 33$. Thus, you can search for exact substrings (a.k.a. k -mers) extracted from the read. The lower the k the more errors your algorithm can handle, but the slower it will become (more locations to explore). To think about:

- Any any k -mer your search for may be broken"(contain a substitution). Even if you find one of several matching regions for a given k -mer, there may be other locations that has a substitution in the reference for this k -mer (i.e., not detected by the given k -mer), but still offer a better scoring location. Thus, you need to try many k -mers per read.
- You must report *all* best mapping locations if there are several best. Python's `in` operator only returns the first match. There are many ways to handle this (continue searching over suffix of the reference starting after each match perhaps). However, Python also has a standard library `re` for more advanced text matching.

Reference coverage

In words, the coverage of a reference sequence is the total number of nucleotides of mapped reads to a reference sequence divided by the sequence length. For example, if N reads of length r mapped uniquely to a given reference sequence of length l , the coverage is simply $\frac{rN}{l}$. However, when we have multiple best locations for a read, we need to weight the read by the number of locations. We also need to account for if the read has multiple locations *within* a reference. We express this mathematically a bit more rigorous below.

Let N be the total number of reads in the sample and M the number of reference sequences. Let m_i be the number of best mapping locations for read i , $i \in [1, n]$, and m_{ij} be the number of (best) mapping locations to reference sequence j (i.e., $m_i = \sum_j m_{ij}$). The coverage C_j of a reference sequence j of length l_j is computed as

$$C_j = \sum_{i=1}^n \frac{r(m_{ij}/m_i)}{l_j}.$$

Output file

The output file should be on the following format `readID, refID, x, e` where `readID` and `refID` are the accessions in the read and reference fasta file, respectively, and `x` is the mapping start position of the read and `e` is the number of substitutions between the read and the reference (i.e., the hamming distance). A read should have exactly m_i lines in this file, one for each best mapping location. Positions should be 0-indexed, just like in Python. For example, in the small dataset (see course webpage), where we have the references

```
>ref1
AAAAAAACCCCCCTTTTTTGGGGGGG
>ref2
TTTTTTGGGGGGGAAAAAACCCCCC
```

and the reads

```
>read1
AAACCCCC
>read2
TTTGGGGG
```

output file 1 should be:

```
read1 , ref1 , 4 , 0
read1 , ref2 , 18 , 0
read2 , ref1 , 18 , 0
read2 , ref2 , 4 , 0
```

Output file 2 (reference coverage) should be:

```
ref1 ,0.3035714285714286  
ref2 ,0.3035714285714286
```

Test data

In the project folder on the course webpage you can find five different datasets. The small dataset has been used in this report and serves as a small example for testing. Then there are four different datasets with 50 reference sequences as various mutation rates w.r.t. each other. For example, the reads5 and refs5 dataset contains 50 references at 5% substitution rate from each other. You can generate your own test data using the script `simulate_dataset.py` available in the project folder.

The example datasets were generated with this script as

```
python simulate_dataset.py refs.fa reads.fa --ref_subs S
```

Where S is the substitution rate (0.0, 0.01, 0.05, 0.10 in our test data). You can tweak the read lengths, number of errors, reference lengths, number of references and number of reads with arguments.

Criteria

This project has the following criteria:

- The program should be generic and capable of handling any fasta files (e.g., one or many reference sequences, one or many reads).
- You are **not** allowed to use any external edit or hamming distance libraries (to estimate distance between strings) for this project. But you can use all module included in the python distribution. Matplotlib (external) should be used for plotting.
- If problems arise during the mapping of a read (e.g., maps outside the border of the reference), an error message should be displayed, but the mapping should continue for other reads.
- Axes should be labeled (e.g., "Count and Nr. best locations") in your figures.
- Your program should save the figure in PDF format with descriptive filename.
- The printed figures should resemble those shown in Figure 1.

For general project requirements and grading criteria, refer to the general instructions at *INDU—Individual Project in Programming*.

The report

As described in the general instructions, the report should focus on the structure of the program, and the text should cover a maximum of three A4 pages. You should include the figures with possible explanations and answers to the questions in the assignment.

Data

You will receive data in the ZIP file `mapping_data.zip` under the project folder on the course website. The data files are three different pairs of reference sequences and reads. In sample1, the references are roughly 90% identity to each other, in sample 2 95% identity, and in sample 3 99%.

Grading

Details in the general instructions in *INDU—Individual Programming Project*.